ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN
KHOA MẠNG MÁY TÍNH VÀ TRUYỀN THÔNG

NT106 — LẬP TRÌNH MẠNG CĂN BẢN

CẨM NANG ĐỒ ÁN LABGUARD

TỔNG QUAN A–Z

HỆ THỐNG GIÁM SÁT VÀ QUẢN TRỊ PHÒNG MÁY TẬP TRUNG

Kiến trúc mạng Client–Server, chuẩn đóng gói dữ liệu và quy trình hiện thực hóa

---

BIÊN SOẠN: VÕ TRỌNG PHÚC

NHÓM THỰC HIỆN: 4 THÀNH VIÊN

TP. HỒ CHÍ MINH — 2026

MỤC LỤC

1 LabGuard giải quyết vấn đề gì? 1
1.1 Bối cảnh thực tế tại phòng thực hành mạng . . . 1
1.2 Đặc thù so với các công cụ kiểm tra thông dụng . . . 2
1.3 So sánh LabGuard với các giải pháp truyền thống . . . 2
1.4 Mục tiêu ra đời của hệ thống LabGuard . . . 3

2 Mô hình OSI và kiến trúc mạng LabGuard 4
2.1 Chi tiết cách LabGuard vận hành trên các tầng mạng . . . 4

3 Ba chương trình chính trong hệ thống LabGuard 5
3.1 LabGuard.Agent (Chương trình máy trạm sinh viên) . . . 6
3.2 LabGuard.Central (Máy chủ trung tâm) . . . 6
3.3 LabGuard.Admin (Màn hình quản trị WinForms) . . . 6

4 Tại sao cả Agent và Admin đều là Client? 6
4.1 Rào cản tường lửa và NAT (Network Address Translation) . . . 7
4.2 Quản lý vòng đời kết nối tập trung . . . 7

5 Tại sao Central đóng vai trò Server? 7
5.1 Mở cổng mạng thụ động (Passive Listening) . . . 7
5.2 Phục vụ đồng thời nhiều kết nối không bị nghẽn . . . 8
5.3 Nơi lưu trữ tập trung dữ liệu an toàn . . . 8

6 Hành trình một thông điệp 8
6.1 Bước 1: Tạo đối tượng dữ liệu trong bộ nhớ C# . . . 9
6.2 Đóng gói đối tượng vào WireEnvelope . . . 9
6.3 Tuần tự hóa đối tượng C# thành mảng byte JSON UTF-8 . . . 10
6.4 Đóng khung nhị phân 4 byte (FrameCodec) . . . 10
6.5 Truyền dòng byte qua Socket TCP tới Cổng 5000 . . . 10
6.6 Tiếp nhận và giải mã tại Central Server . . . 10
6.7 Xác thực và kích hoạt cảnh báo sự cố . . . 10

7 Phân hệ Protocol hiện đã làm được gì? 11
7.1 Cấu trúc vỏ bọc dữ liệu chuẩn (WireEnvelope) . . . 11
7.2 Bộ kiểm tra tính hợp lệ của bản tin (EnvelopeValidator) . . . 12
7.3 Bộ đóng khung nhị phân (FrameCodec) . . . 12
7.4 Thuật toán giải mã tích lũy (ReadExactlyBytesAsync) . . . 12
7.5 Danh mục 16 Fixture JSON quy chuẩn . . . 13

8 Hiện trạng mã nguồn thực tế của dự án 14

9 Bản đồ tệp tin hiện tại và Thiết kế CSDL 15
9.1 Các bảng/chức năng CSDL đã được chốt ở mức kiến trúc . . . 15

10 Phương pháp 5 câu hỏi đọc hiểu code C# 16

11 Bốn thành viên phụ trách những phần việc gì? 16

12 Hành trình phát triển theo 6 chặng công việc **17**

13 Quy trình làm việc nhóm an toàn với Git **18**

14 Cách biên dịch và kiểm thử dự án hiện tại **18**

15 Luồng hoạt động toàn bộ đồ án **19**
15.1 Thu thập số liệu định kỳ và cập nhật Dashboard 19
15.2 Phát hiện mất kết nối mạng (Liveness Timeout) 19
15.3 Phát hiện quá tải CPU và tự phục hồi (RuleEngine) 20
15.4 Giải quyết xung đột khóa lạc quan (OCC Conflict Resolution) 20

16 Bảng tra cứu thuật ngữ mạng cốt lõi **21**

17 Bộ câu hỏi vấn đáp bảo vệ đồ án NT106 **22**

1. LabGuard giải quyết vấn đề gì?

TÓM TẮT 30 GIÂY

- Phòng máy 40–50 máy tính thường gặp sự cố: máy đơ cứng, rút lỏng cáp mạng, tắt nhầm phần mềm thi.
- Giảng viên không thể đi bộ kiểm tra thủ công từng máy; Ping chỉ phản ánh khả năng liên lạc cơ bản. LabGuard gom nhiều loại thông tin của phòng máy vào cùng một hệ thống giám sát.
- LabGuard giải quyết bài toán: giám sát CPU/RAM thời gian thực, tự động phát hiện mất kết nối sau 20s.
- Cung cấp bảng điều khiển WinForms hiển thị bản đồ trực quan có màu sắc cảnh báo.

1.1 Bối cảnh thực tế tại phòng thực hành mạng

Hãy hình dung một buổi chiều thi thực hành môn học tại phòng máy của Trường Đại học Công nghệ Thông tin (UIT). Phòng thực hành có quy mô khoảng 40 đến 50 máy tính sinh viên hoạt động đồng thời dưới sự giám sát của một cán bộ quản trị hoặc giảng viên coi thi:

- Lấy ví dụ chiếc máy **`LAB01-PC02`** ở dãy bàn thứ hai: một sinh viên đang làm bài thi vô tình chạy một đoạn chương trình có vòng lặp vô tận. Mức tải CPU của `LAB01-PC02` vọt lên 94% và ngốn sạch RAM, khiến máy tính bị đơ chuột, sinh viên hốt hoảng không biết xử lý ra sao.
- Một máy tính khác ở góc phòng bị lỏng chân cáp mạng RJ45, mất kết nối âm thầm khiến bài thi không thể gửi lên hệ thống nộp bài của trường.
- Một sinh viên tò mò tắt nhầm dịch vụ phần mềm thi cử cần thiết đang chạy ngầm.
- Giảng viên coi thi không có cách nào biết được chính xác máy nào đang gặp sự cố trừ khi trực tiếp đi bộ đến từng bàn máy để kiểm tra thủ công.

Trong môi trường phòng thực hành tập trung, việc quản lý thủ công từng máy trạm bộc lộ hàng loạt nhược điểm: tốn thời gian, phản ứng chậm trễ khi có sự cố mạng, và thiếu hụt dữ liệu lịch sử để phân tích nguyên nhân gốc rễ.

Hình này chỉ cần nhìn luồng thông tin từ trái sang phải: các máy trạm sinh viên gửi dữ liệu về máy chủ Central trên cổng 5000, Central lưu CSDL và đẩy cảnh báo sang Admin.

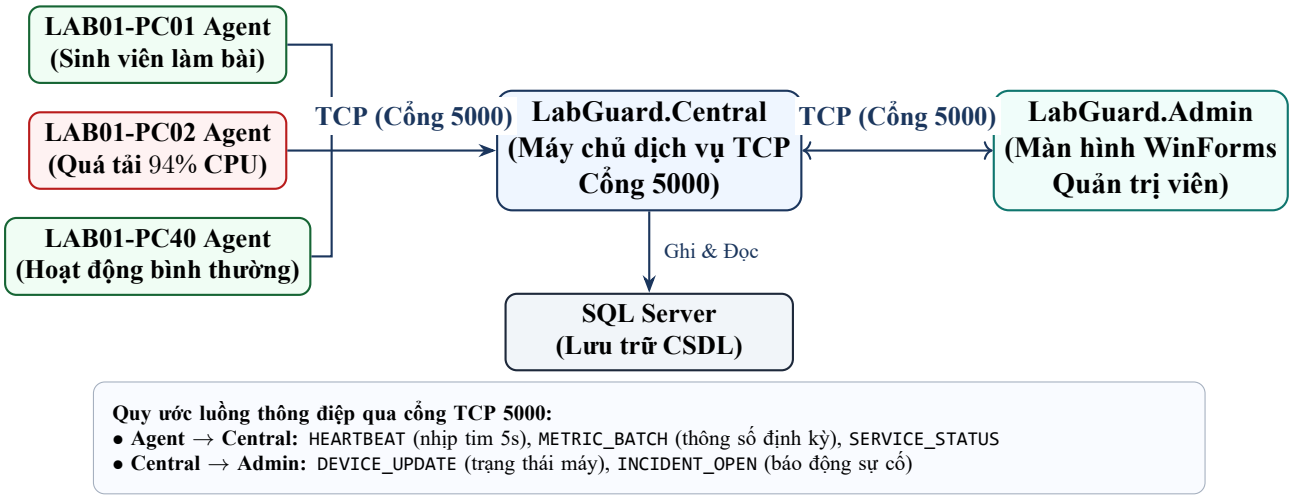


Figure 1: Mô hình tổng quan bài toán giám sát phòng máy thực hành UIT

Ý chính: Toàn bộ giao tiếp mạng trong LabGuard đều hội tụ về máy chủ trung tâm Central Server trên cổng 5000 duy nhất, tạo điểm tiếp nhận tập trung cho cả phòng máy.

1.2 Đặc thù so với các công cụ kiểm tra thông dụng

Trước khi bắt tay xây dựng LabGuard, ta xem xét sự khác biệt giữa các công cụ quản trị mạng thông dụng và yêu cầu thực tế của phòng máy:

1. **Lệnh Ping (ICMP Echo):** Ping chỉ phản ánh card mạng có phản hồi tín hiệu hay không. Ping không thể cho biết máy tính có đang bị đơ chuột, quá tải CPU/RAM, hay phần mềm thi cử bị tắt ngang hay không.
2. **Giao thức SNMP (Simple Network Management Protocol):** SNMP là tiêu chuẩn công nghiệp rất mạnh mẽ cho thiết bị mạng chuyên dụng (như Router/Switch). Tuy nhiên trong phạm vi môn học Lập trình mạng, mục tiêu của đồ án là tự thiết kế giao thức ứng dụng trên nền TCP và làm chủ toàn bộ tầng socket, thay vì phụ thuộc dịch vụ SNMP có sẵn trên hệ điều hành.
3. **Giao thức SSH / Telnet:** Đòi hỏi mở cổng lắng nghe thụ động trên từng máy sinh viên, làm tăng nguy cơ quét cổng trái phép giữa các máy trong phòng thi.
4. **Các giải pháp đám mây thương mại:** Thường đòi hỏi kết nối Internet băng thông liên tục và chi phí bản quyền, không phù hợp với mạng nội bộ LAN cách ly của phòng thực hành.

1.3 So sánh LabGuard với các giải pháp truyền thống

Bảng dưới đây tổng hợp các tiêu chí kỹ thuật giữa LabGuard và các phương thức thông dụng:

Table 1: Bảng so sánh kỹ thuật giữa LabGuard và các công cụ giám sát khác

Tiêu chí	Ping/ICMP	SNMP v2c/v3	SSH/WinRM	LabGuard NT106
Tầng mạng hoạt động	Tầng 3 (IP)	Tầng 7 (UDP)	Tầng 7 (TCP)	Tầng 7 (TCP tự định nghĩa)
Vượt tường lửa máy trạm	Có (ICMP)	Khó (mở UDP in)	Rất khó (mở TCP in)	Dễ dàng (Agent gọi ra ngoài)
Đo đạc CPU/RAM sâu	Không	Có	Có	Có (thu thập thời gian thực)
Độ trễ phát hiện sự cố	Chậm (thăm dò)	Trung bình	Chậm (nặng)	Nhanh (Heartbeat 5s)
Tính học thuật môn mạng	Rất thấp	Trung bình	Thấp	Tối đa (tự viết Socket, Framing)

1.4 Mục tiêu ra đời của hệ thống LabGuard

Đồ án LabGuard ra đời nhằm giải quyết triệt để bài toán trên bằng cách xây dựng một hệ thống chuyên biệt, gọn nhẹ, hoàn toàn tự chủ trong mạng nội bộ theo đúng chuẩn mực môn Lập trình mạng:

1. **Giám sát thời gian thực:** Định kỳ thu thập các chỉ số phần cứng cốt lõi (CPU, RAM khả dụng, tiến trình) từ máy trạm gửi về máy chủ.
2. **Phát hiện mất kết nối tự động:** Thông qua nhịp tim định kỳ 5 giây, nếu máy trạm im lặng quá 20 giây (> 20s), máy chủ tự động kích hoạt cảnh báo mất mạng.
3. **Động cơ cảnh báo thông minh (Rule Engine):** Tự động phát hiện hiện tượng quá tải phần cứng (ví dụ máy `LAB01-PC02` có CPU vượt ngưỡng 90% trong 3 chu kỳ liên tiếp) để cảnh báo tức thời lên màn hình quản trị.
4. **Bảng điều khiển trực quan (Admin Dashboard):** Giao diện đồ họa WinForms hiển thị bản đồ toàn bộ phòng máy với màu sắc rõ ràng.
5. **Tính toàn vẹn dữ liệu (Khóa lạc quan OCC):** Khi nhiều quản trị viên cùng thao tác, hệ thống ngăn chặn ghi đè mâu thuẫn dữ liệu.

SAU PHẦN NÀY, BẠN CHỈ CẦN NHỚ:

- LabGuard giám sát phòng máy theo cơ chế Client–Server trên nền TCP.
- Máy trạm tự chủ gọi ra máy chủ qua cổng 5000, dễ dàng vượt qua tường lửa của Windows.

2. Mô hình OSI và kiến trúc mạng LabGuard

TÓM TẮT 30 GIÂY

- LabGuard bao phủ trọn vẹn từ Tầng 1 đến Tầng 7 của mô hình OSI.
- Trọng tâm học phần NT106 nằm ở Tầng 4 (TCP Socket, Port 5000) và Tầng 6/7 (Framing, JSON).
- Đóng khung nhị phân 4 byte Big-Endian (`FrameCodec`) giải quyết dứt điểm hiện tượng dính/xé gói TCP.

Để hiểu tường tận đồ án Lập trình mạng căn bản, chúng ta đối chiếu kiến trúc LabGuard vào mô hình 7 tầng chuẩn của OSI:

Hình này chỉ cần nhìn đối chiếu từng cặp tầng: bên trái là tên tầng chuẩn OSI, bên phải là thành phần kỹ thuật tương ứng trong LabGuard.

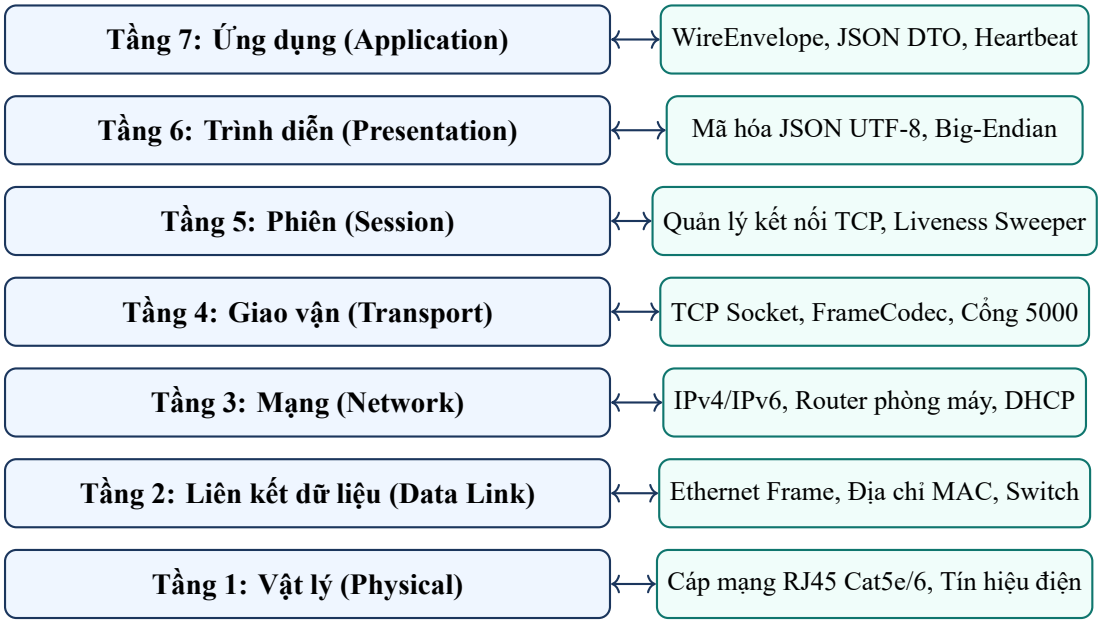


Figure 2: Đối chiếu phân hệ LabGuard với 7 tầng mô hình OSI

Ý chính: Đồ án tập trung hiện thực các kỹ thuật từ tầng 4 đến tầng 7: mở socket TCP, đóng khung tin nhị phân, giải mã chuỗi JSON và xử lý logic nghiệp vụ.

2.1 Chi tiết cách LabGuard vận hành trên các tầng mạng

1. **Tầng 1 & 2 (Physical & Data Link):** Các máy tính trong phòng thực hành kết nối với nhau qua cáp mạng RJ45 và Switch 1 Gbps.
2. **Tầng 3 (Network):** Mỗi máy trạm nhận một địa chỉ IPv4 nội bộ (`192.168.1.x`). Máy chủ Central duy trì IP tĩnh cố định để các máy con tìm đến.
3. **Tầng 4 (Transport):** LabGuard dùng TCP tin cậy trên cổng 5000 duy nhất. Bộ đóng khung nhị phân `FrameCodec` nằm ở ranh giới giữa tầng 4 và tầng 5, giúp biến dòng byte vô tận của TCP thành các bức thông điệp độc lập.

4. **Tầng 5 (Session):** Central Server duy trì danh mục các kết nối đang mở qua `ConcurrentDictionary`. Luồng quét `LivenessMonitor` định kỳ kiểm tra phiên kết nối.
5. **Tầng 6 (Presentation):** Dữ liệu C# được chuyển đổi thành chuỗi JSON UTF-8. Độ dài khung tin được ghi bằng 4 byte số nguyên chuẩn Big-Endian.
6. **Tầng 7 (Application):** Cấu trúc phong bì `WireEnvelope` và danh mục thông điệp `MessageTypes.cs` quy định hành vi trao đổi thông tin của toàn hệ thống.

SAU PHẦN NÀY, BẠN CHỈ CẦN NHỚ:

- Nắm vững vai trò từng tầng giúp bạn tự tin trả lời vấn đáp khi thầy cô chỉ vào mô hình OSI.
- Tiêu đề 4 byte Big-Endian thuộc tầng 6 (Presentation) nhằm chuẩn hóa dữ liệu trước khi truyền xuống TCP (tầng 4).

3. Ba chương trình chính trong hệ thống LabGuard

TÓM TẮT 30 GIÂY

- Hệ thống gồm 3 chương trình độc lập: Agent (máy sinh viên), Central (máy chủ), Admin (màn hình quản trị).
- Cả 3 chương trình đều tham chiếu và sử dụng chung thư viện `LabGuard.Protocol`.
- Agent và Admin đều đóng vai trò TCP Client kết nối về Central trên cổng 5000.

Kiến trúc phần mềm của LabGuard được chia thành ba chương trình độc lập, hoạt động phối hợp nhịp nhàng với nhau thông qua mạng TCP/IP:

Hình này chỉ cần nhìn theo các đường kết nối: Agent và Admin kết nối TCP về Central (cổng 5000), và cả ba cùng sử dụng chung thư viện Protocol bên dưới.

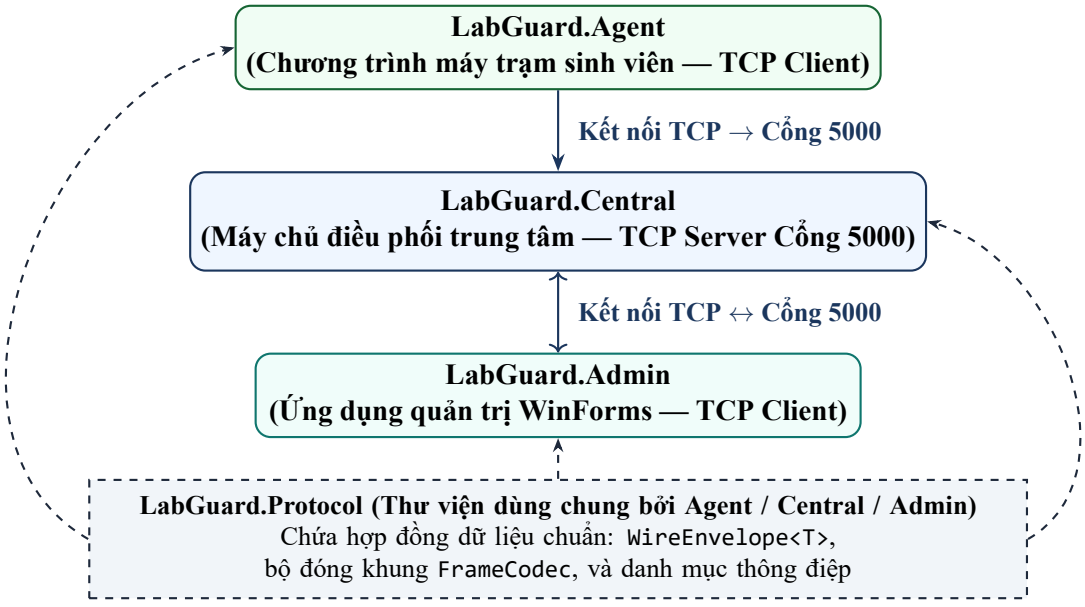


Figure 3: Mối quan hệ kết nối giữa 3 chương trình trong LabGuard

Ý chính: Nhóm dùng chung một thư viện Protocol duy nhất để bảo đảm tất cả các thành viên đều tuân thủ cùng một quy chuẩn đóng gói mạng.

3.1 LabGuard.Agent (Chương trình máy trạm sinh viên)

- **Bản chất:** Ứng dụng chạy ngầm trên từng máy tính sinh viên, không chiếm dụng màn hình làm bài.
- **Cơ chế hoạt động:** Là TCP Client chủ động mở kết nối ra Central Server tại cổng 5000.
- **Nhiệm vụ chính:** Định kỳ 5 giây đo CPU, RAM và gửi nhịp tim `HEARTBEAT`; lưu đệm vào `ReplayBuffer` khi rớt mạng để gửi bù sau.

3.2 LabGuard.Central (Máy chủ trung tâm)

- **Bản chất:** Trái tim và bộ não điều phối của toàn bộ hệ thống phòng máy.
- **Cơ chế hoạt động:** Là TCP Server lắng nghe trên cổng 5000 duy nhất, tiếp nhận đồng thời cả Agent và Admin.
- **Nhiệm vụ chính:** Lưu dữ liệu vào SQL Server, chạy động cơ kiểm tra sống còn (`LivenessMonitor`) và động cơ quy tắc (`RuleEngine`) để phát hiện máy hỏng.

3.3 LabGuard.Admin (Màn hình quản trị WinForms)

- **Bản chất:** Ứng dụng đồ họa Desktop (Windows Forms) dành cho giám sát viên.
- **Cơ chế hoạt động:** Là TCP Client kết nối tới Central Server trên cổng 5000 để lấy số liệu.
- **Nhiệm vụ chính:** Hiển thị danh sách máy tính phòng thi, danh sách sự cố thời gian thực và cung cấp nút bấm để xử lý sự cố.

SAU PHẦN NÀY, BẠN CHỈ CẦN NHỚ:

- Thư viện `LabGuard.Protocol` là hợp đồng bất biến gắn kết 3 phân hệ lại với nhau.
- Phân chia rõ ràng: Bạn A làm Agent, Bạn B làm Central, Bạn C làm Admin, Bạn D làm Gateway/Thử tải.

4. Tại sao cả Agent và Admin đều là Client?

TÓM TẮT 30 GIÂY

- Máy sinh viên và máy giảng viên thường nằm sau tường lửa Windows Firewall và mạng NAT.
- Tường lửa mặc định chặn kết nối đi vào (Inbound) nhưng cho phép kết nối đi ra (Outbound).
- Agent và Admin đóng vai trò Client chủ động gọi ra ngoài, vượt tường lửa dễ dàng không cần mở cổng.

Một trong những câu hỏi kiến trúc cốt lõi mà giảng viên chấm đồ án NT106 luôn chất vấn sinh viên là: *"Tại sao chương trình Agent chạy trên máy sinh viên và chương trình Admin chạy trên máy giảng viên đều được thiết kế là TCP Client, mà không phải là TCP Server?"*

4.1 Rào cản tường lửa và NAT (Network Address Translation)

- Máy tính sinh viên và laptop của giảng viên thường nhận địa chỉ IP động thông qua DHCP và nằm sau tường lửa Windows Firewall.
- Windows Firewall mặc định chặn tất cả các yêu cầu kết nối đi vào (Inbound), nhưng cho phép thoải mái các kết nối đi ra ngoài (Outbound).
- Nếu Agent là Server (mở cổng chờ trên máy sinh viên), Central muốn lấy số liệu sẽ phải chủ động kết nối vào từng máy. Kết nối này lập tức bị tường lửa của Windows chặn đứng!
- Khi Agent là Client, nó chủ động mở kết nối đi ra ngoài tới IP tĩnh của Central Server. Kết nối này đi qua tường lửa dễ dàng mà không cần cấu hình phức tạp trên từng máy sinh viên.

4.2 Quản lý vòng đời kết nối tập trung

- Khi máy sinh viên bị tắt nguồn hoặc rút cáp mạng, chỉ bản thân máy đó bị ảnh hưởng. Nếu Central là Client đi kết nối tới 50 Server máy trạm, Central sẽ phải liên tục thăm dò (polling) 50 IP khác nhau, gây lãng phí băng thông mạng cực lớn.
- Đặt Central làm Server tập trung giúp toàn hệ thống chỉ có một địa chỉ IP tĩnh duy nhất. Tất cả máy trạm chỉ việc kết nối về máy chủ khi khởi động.

SAU PHẦN NÀY, BẠN CHỈ CẦN NHỚ:

- Thực thể cố định, tài nguyên mạnh, phục vụ nhiều máy → đóng vai trò **Server**.
- Thực thể hay bật/tắt, nằm sau tường lửa → đóng vai trò **Client**.

5. Tại sao Central đóng vai trò Server?

TÓM TẮT 30 GIÂY

- Central mở cổng thụ động (Passive Open) trên cổng 5000 ngồi chờ các máy Client kết nối tới.
- Dùng lập trình bất đồng bộ (`async/await`) để phục vụ hàng chục máy cùng lúc mà không tốn RAM.
- Là nơi duy nhất giữ chuỗi kết nối CSDL SQL Server, bảo đảm an toàn dữ liệu phòng máy.

5.1 Mở cổng mạng thụ động (Passive Listening)

Trong mô hình Socket TCP, Central Server thực hiện 4 bước kinh điển:

1. **Tạo Socket:** Khởi tạo Socket hỗ trợ IPv4 và giao thức hướng dòng TCP.
2. **Gắn kết cổng (Bind):** Gắn Socket với địa chỉ `IPAddress.Any` và cổng 5000 (Central TCP Endpoint).

3. **Lắng nghe (Listen):** Đưa socket vào trạng thái sẵn sàng đón nhận kết nối.
4. **Chấp nhận kết nối (Accept):** Vòng lặp `AcceptTcpClientAsync()` tiếp nhận các lượt bắt tay 3 bước từ Client.

5.2 Phục vụ đồng thời nhiều kết nối không bị nghẽn

Central Server sử dụng cơ chế lập trình bất đồng bộ hiện đại (`async/await` trong .NET):

- Không tạo một Thread mới cho mỗi máy (tránh làm tràn bộ nhớ và quá tải CPU do Context Switch).
- Tận dụng cơ chế I/O Completion Ports (IOCP) của Windows, cho phép một nhóm nhỏ luồng trong ThreadPool phục vụ hàng trăm kết nối mạng cùng lúc.

5.3 Nơi lưu trữ tập trung dữ liệu an toàn

Central Server là thực thể duy nhất kết nối trực tiếp với CSDL SQL Server:

- Máy sinh viên không bao giờ được cấp tài khoản CSDL, ngăn ngừa sinh viên tò mò phá hoại dữ liệu.
- Central tối ưu hiệu năng ghi đĩa bằng cách gom các bản ghi thành từng lô (Batch Insert).

SAU PHẦN NÀY, BẠN CHỈ CẦN NHỚ:

- Central Server là nguồn chân lý duy nhất (Single Source of Truth) quyết định trạng thái của phòng máy.
- Mọi kết nối của Agent và Admin đều quy về cổng TCP 5000 của Central Server.

6. Hành trình một thông điệp đi từ Agent tới Central

TÓM TẮT 30 GIÂY

- Theo dõi hành trình cụ thể của máy **`LAB01-PC02`** khi tải CPU vọt lên 94%.
- Đi qua 7 bước: Đo đạc → Đóng phong bì → Tuần tự hóa JSON → Thêm 4 byte Big-Endian → Truyền TCP → Giải mã → Đánh giá quy tắc & Lưu CSDL.
- Nắm vững 7 bước này giúp bạn trả lời trôi chảy bất kỳ câu hỏi luồng dữ liệu nào từ ban giám khảo.

Hãy theo dõi hành trình của một gói tin thông số máy trạm (`METRIC_BATCH`) phát sinh từ chiếc máy **`LAB01-PC02`** khi CPU bị quá tải đạt 94% cho tới khi được lưu vào cơ sở dữ liệu và hiển thị lên màn hình Admin:

Hình này chỉ cần nhìn theo các bước từ trên xuống dưới: từ lúc sinh ra trong bộ nhớ C# của máy trạm cho đến khi được lưu vào CSDL máy chủ.

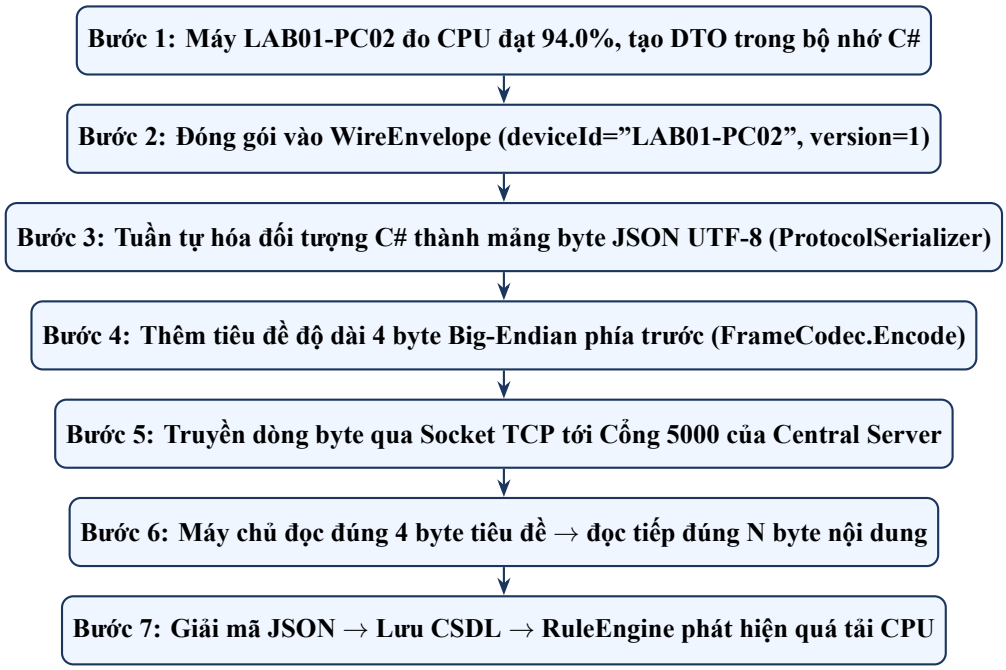


Figure 4: Hành trình tuần tự 7 bước của thông điệp từ LAB01-PC02 tới Central

Ý chính: Dữ liệu luôn được đóng gói chặt chẽ qua hai lớp bảo vệ: phong bì JSON ở tầng ứng dụng và tiêu đề 4 byte ở tầng giao vận.

6.1 Bước 1: Tạo đối tượng dữ liệu trong bộ nhớ C#

Dịch vụ thu thập số liệu trên máy `LAB01-PC02` phát hiện CPU vọt lên 94.0%, RAM khả dụng còn 1,024 MB. Dữ liệu được gán vào một đối tượng C#:

ĐOẠN NÀY CẦN NHÌN ĐIỀU GÌ?

- Mẫu đo ghi nhận giá trị CPU 94.0% và đơn vị tính tương ứng.
- Dữ liệu nội dung thuần túy trước khi được đặt vào phong bì mạng.

```
var sample = new MetricSampleDto
{
    MetricName = "CPU",
    MetricValue = 94.0,
    Unit = "%",
    RecordedAtUtc = DateTime.UtcNow.ToString("O")
};
```

6.2 Đóng gói đối tượng vào WireEnvelope

Gói dữ liệu thô trên được đặt vào phong bì 6 trường quy chuẩn với định danh máy `LAB01-PC02`:

ĐOẠN NÀY CẦN NHÌN ĐIỀU GÌ?

- `deviceId` được gán chính xác là `"LAB01-PC02"`.
- Hàm tĩnh `Create()` tự động sinh UUID v4 và lấy giờ UTC chuẩn quốc tế.

```
var envelope = WireEnvelope<MetricBatchPayload>.Create(
    MessageTypes.MetricBatch,
    "LAB01-PC02",
    new MetricBatchPayload { Samples = [sample] });
```

6.3 Tuần tự hóa đối tượng C# thành mảng byte JSON UTF-8

Đối tượng `WireEnvelope` được chuyển đổi thành chuỗi JSON UTF-8 (ví dụ kích thước 185 byte):

ĐOẠN NÀY CẦN NHÌN ĐIỀU GÌ?

- Chuỗi JSON được chuyển sang mảng byte nhị phân UTF-8 qua `ProtocolSerializer`.

```
byte[] jsonBytes = ProtocolSerializer.Serialize(envelope);
// jsonBytes.Length == 185 byte
```

6.4 Đóng khung nhị phân 4 byte (FrameCodec)

Để giải quyết triệt để hiện tượng dính/xé gói TCP, `FrameCodec` ghép 4 byte độ dài Big-Endian vào đầu mảng byte JSON:

Hình này chỉ cần nhìn vào hai phần của khung tin: 4 byte tiêu đề màu đỏ bên trái chỉ độ dài, và mảng byte JSON màu xanh bên phải chứa nội dung.

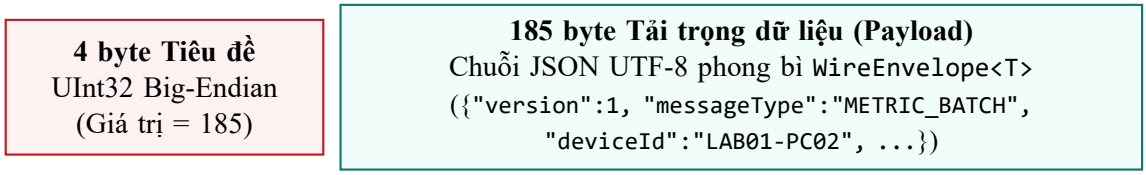


Figure 5: Cấu trúc một khung tin nhị phân chuẩn trên đường truyền mạng

Ý chính: Nhờ có 4 byte đầu tiên này, phía nhận luôn biết chính xác cần đọc bao nhiêu byte nữa thì trọn vẹn một gói tin.

6.5 Truyền dòng byte qua Socket TCP tới Cổng 5000

Mảng 189 byte được ghi vào `NetworkStream.WriteAsync()` gửi sang Central Server trên cổng 5000.

6.6 Tiếp nhận và giải mã tại Central Server

Tại Central Server, phương thức `ReadExactlyBytesAsync` đọc đúng 4 byte đầu tiên để biết độ dài là 185, kiểm tra an toàn ($185 \leq 256$ KiB) rồi đọc tiếp đúng 185 byte JSON.

6.7 Xác thực và kích hoạt cảnh báo sự cố

1. `EnvelopeValidator` xác nhận thông điệp hợp lệ.
2. Số liệu CPU 94.0% của `LAB01-PC02` được lưu vào bảng `MetricSamples` trong SQL Server (sau khi B-005 hiện thực CSDL).

3. `RuleEngine` kiểm tra: nếu `LAB01-PC02` duy trì CPU $\geq 90\%$ trong 3 chu kỳ đo liên tiếp (3 lần gửi 5s), Central kích hoạt sự cố `HIGH_CPU`, gửi thông báo `INCIDENT_OPEN` sang Admin. Màn hình Dashboard lập tức tô đỏ dòng máy `LAB01-PC02`!

SAU PHẦN NÀY, BẠN CHỈ CẦN NHỚ:

- 7 bước diễn ra trong chưa đầy 10 mili-giây trên mạng LAN phòng thực hành.
- Luôn nhớ: Dữ liệu thô → WireEnvelope → JSON UTF-8 → 4 byte Big-Endian.

7. Phân hệ Protocol hiện đã làm được gì?

TÓM TẮT 30 GIÂY

- Thư viện `LabGuard.Protocol` đã hoàn thành 100% và toàn bộ bài test tự động đã PASS.
- Định nghĩa phong bì `WireEnvelope` 6 trường cố định và bộ đóng khung `FrameCodec` 4 byte.
- Quy định danh mục 69 loại thông điệp và 16 fixture JSON đại diện cho toàn bộ hệ thống.

7.1 Cấu trúc vỏ bọc dữ liệu chuẩn (WireEnvelope)

Tệp `src/LabGuard.Protocol/WireEnvelope.cs` định nghĩa lớp phong bì dữ liệu chứa đúng 6 trường bắt buộc:

ĐOẠN NÀY CẦN NHÌN ĐIỀU GÌ?

- 6 trường chuẩn: `version`, `messageType`, `requestId`, `deviceId`, `sentAtUtc`, `payload`.
- Hàm tĩnh `Create()` giúp tự động điền các trường hệ thống.

```
// Trich doan tu ma nguon thuc te: 6 thuoc tinh phong bi du lieu chuan
public class WireEnvelope<T>
{
    [JsonPropertyName("version")]     public int Version { get; set; } = 1;
    [JsonPropertyName("messageType")] public string MessageType { get; set; } =
        string.Empty;
    [JsonPropertyName("requestId")]   public string RequestId { get; set; } =
        string.Empty;
    [JsonPropertyName("deviceId")]    public string DeviceId { get; set; } =
        string.Empty;
    [JsonPropertyName("sentAtUtc")]   public string SentAtUtc { get; set; } =
        string.Empty;
    [JsonPropertyName("payload")]     public T Payload { get; set; } = default!;

    // Phuong thuc Create(...) tien ich khoi tao nhanh (xem file nguon day du)
    public static WireEnvelope<T> Create(string type, string devId, T payload,
        string? reqId = null) => ...;
}
```

Listing 1: Ví dụ rút gọn dựa trên mã nguồn hiện tại: src/LabGuard.Protocol/WireEnvelope.cs

7.2 Bộ kiểm tra tính hợp lệ của bản tin (EnvelopeValidator)

Tệp `src/LabGuard.Protocol/EnvelopeValidator.cs` kiểm tra nghiêm ngặt: phiên bản phải bằng 1, tên thông điệp thuộc 69 loại chuẩn, RequestId là chuỗi GUID hợp lệ, DeviceId không rỗng, và thời gian có hậu tố UTC.

7.3 Bộ đóng khung nhị phân (FrameCodec)

Tệp `src/LabGuard.Protocol/FrameCodec.cs` bổ sung 4 byte độ dài Big-Endian với hạn mức tối đa 256 KiB:

ĐOẠN NÀY CẦN NHÌN ĐIỀU GÌ?

- `BinaryPrimitives.WriteUInt32BigEndian` bảo đảm thứ tự byte mạng chuẩn mực.
- Giới hạn `MaxPayloadSize = 262_144` (256 KiB) phòng chống tấn công tràn RAM.

```
// Trich doan: Hang so co dinh 4 byte do dai va gioi han 256 KiB
public static class FrameCodec
{
    public const int HeaderSize = 4;                                    // 4 byte do dai Big-Endian
    public const int MaxPayloadSize = 262_144;                          // 256 KiB
    public const int MaxWireFrameSize = HeaderSize + MaxPayloadSize; // 262,148 bytes
}
```

Listing 2: Ví dụ rút gọn dựa trên mã nguồn hiện tại: Hằng số giới hạn trong FrameCodec.cs

ĐOẠN NÀY CẦN NHÌN ĐIỀU GÌ?

- `BinaryPrimitives.WriteUInt32BigEndian` ghi đúng thứ tự byte mạng chuẩn mực.
- Nối liền 4 byte độ dài với mảng byte nội dung JSON phía sau.

```
// Trich doan: Dong goi 4 byte Big-Endian vao dau mang byte
public static byte[] Encode(ReadOnlyMemory<byte> payload)
{
    if (payload.Length == 0 || payload.Length > MaxPayloadSize)
        throw new ArgumentException("Kich thuoc payload khong hop le.");

    byte[] frame = new byte[HeaderSize + payload.Length];
    BinaryPrimitives.WriteUInt32BigEndian(frame.AsSpan(0, HeaderSize), (uint)payload.Length);
    payload.Span.CopyTo(frame.AsSpan(HeaderSize));
    return frame;
}
```

Listing 3: Ví dụ rút gọn dựa trên mã nguồn hiện tại: Phương thức Encode trong FrameCodec.cs

7.4 Thuật toán giải mã tích lũy (ReadExactlyBytesAsync)

Phương thức `ReadExactlyBytesAsync` dùng vòng lặp tích lũy để bảo đảm gom đủ từng byte:

ĐOẠN NÀY CẦN NHÌN ĐIỀU GÌ?

- Vòng lặp `while (totalRead < count)` tiếp tục đọc cho tới khi nhận đủ số byte.
- Báo lỗi ngắt kết nối an toàn nếu socket đóng trước khi nhận đủ.

```
// Trich doan: Vong lap gom tich luy du count byte tu NetworkStream
public static async Task ReadExactlyBytesAsync(
    Stream stream, byte[] buffer, int offset, int count, CancellationToken ct =
        default)
{
    int totalRead = 0;
    while (totalRead < count)
    {
        int read = await stream.ReadAsync(buffer.AsMemory(offset + totalRead,
            count - totalRead), ct);
        if (read == 0)
            throw new EndOfStreamException($"Mat ket noi TCP: da doc
                {totalRead}/{count} bytes.");
        totalRead += read;
    }
}
```

Listing 4: Ví dụ rút gọn dựa trên mã nguồn hiện tại: ReadExactlyBytesAsync trong FrameCodec.cs

7.5 Danh mục 16 Fixture JSON quy chuẩn

Nhiệm vụ A-001 đã đóng băng 16 mẫu bản tin JSON đại diện cho các luồng tương tác cốt lõi nhất:

Table 2: Danh mục 16 Fixture JSON đại diện đóng băng tại nhiệm vụ A-001

Tên Fixture	Chiều gửi	Cổng	Mục đích nghiệp vụ
HEARTBEAT	Agent → Central	5000	Nhịp tim định kỳ 5 giây chứng minh máy còn hoạt động.
HEARTBEAT_ACK	Central → Agent	5000	Máy chủ phản hồi đã nhận nhịp tim.
METRIC_BATCH	Agent → Central	5000	Gửi số liệu CPU, RAM của máy trạm định kỳ.
BATCH_ACK	Central → Agent	5000	Máy chủ xác nhận đã lưu lô số liệu đo đạc.
SERVICE_STATUS	Agent → Central	5000	Báo cáo trạng thái tiến trình phần mềm trọng yếu.
HELLO_AGENT	Agent → Central	5000	Máy trạm chào hỏi khi mở kết nối ban đầu.
HELLO_ACK	Central → Client	5000	Máy chủ xác nhận đối tác kết nối thành công.
AUTH_AGENT	Agent → Central	5000	Máy trạm gửi thông tin xác thực định danh.
AUTH_RESULT	Central → Agent	5000	Máy chủ thông báo kết quả xác thực.
AGENT_GOODBYE	Agent → Central	5000	Máy trạm chủ động ngắt kết nối an toàn.
GOODBYE_ACK	Central → Agent	5000	Máy chủ xác nhận đóng phiên êm đẹp.
LOGIN	Admin → Central	5000	Quản trị viên gửi tài khoản đăng nhập.
LOGIN_RESULT	Central → Admin	5000	Máy chủ phản hồi kết quả và vai trò (RBAC).
INCIDENT_ACK	Admin → Central	5000	Quản trị viên tiếp nhận sự cố (kèm khóa OCC).
COMMAND_RESULT	Central → Admin	5000	Máy chủ phản hồi kết quả lệnh quản trị (thành công hoặc xung đột).
ERROR	Hai chiều	5000	Thông báo lỗi giao thức chuẩn hóa.

SAU PHẦN NÀY, BẠN CHỈ CẦN NHỚ:

- Thư viện Protocol là tài sản chung đã hoàn thành 100% (Protocol tests PASS).
- Cả 3 thành viên B, C, D đều dựa vào thư viện này để xây dựng phân hệ của mình.

8. Hiện trạng mã nguồn thực tế của dự án

TÓM TẮT 30 GIÂY

- Thư viện `Protocol` (A-001 & A-002) đã hoàn thành 100% (toàn bộ kiểm thử tự động đã PASS).
- Phân hệ Agent đã hoàn thành A-003 (`AgentSocketClient.cs`, 16/16 tests PASS); các chặng A-004+ chưa triển khai để sinh viên A tự tay viết.
- Các phân hệ Central, Admin hiện là khung sườn tối thiểu để sinh viên B, C tự tay lập trình.

Table 3: Bảng hiện trạng mã nguồn thực tế của dự án LabGuard

Phân hệ / Thư mục	Hiện trạng thực tế	Mô tả chi tiết và phân công
src/LabGuard.Protocol/	**ĐÃ HOÀN THÀNH (100%)**	Chứa `WireEnvelope`, `FrameCodec`, `EnvelopeValidator`, các DTO và bộ kiểm thử giao thức đã PASS.
src/LabGuard.Agent/	**A-003 ĐÃ XONG**	Chứa `AgentSocketClient.cs` (BoundedChannel 256, Single-Writer Loop, `CreateOutbox` factory, DisposeAsync đúng) và `Program.cs` khung sườn. Kiểm thử A-003: 16/16 PASS. A-004+ do Sinh viên A thực hiện.
src/LabGuard.Central/	**CHƯA LÀM / DỰ KIẾN**	Hiện chỉ chứa file `Program.cs` giữ chỗ. Bạn B sẽ tự tay viết server TCP, CSDL và RuleEngine.
src/LabGuard.Admin/	**CHƯA LÀM / DỰ KIẾN**	Hiện chỉ chứa file `Program.cs` và form rỗng. Bạn C sẽ tự tay thiết kế bảng điều khiển WinForms.
src/LabGuard.Simulator/	**CHƯA LÀM / DỰ KIẾN**	Hiện chỉ chứa file `Program.cs` rỗng. Bạn D sẽ viết công cụ tạo máy ảo thử tải và bắt gói Wireshark.
tests/LabGuard.Protocol.Tests/	**ĐÃ HOÀN THÀNH (100%)**	Bộ kiểm thử tự động giao thức đã PASS toàn diện.

SAU PHẦN NÀY, BẠN CHỈ CẦN NHỚ:

- Bạn hoàn toàn yên tâm vì mọi phần việc chưa làm đều có hướng dẫn chi tiết trong cẩm nang từng thành viên.
- Hãy tự tay gõ từng dòng lệnh để hiểu sâu bản chất mạng máy tính.

9. Bản đồ tệp tin hiện tại và Thiết kế CSDL

TÓM TẮT 30 GIÂY

- File khởi đầu nhanh duy nhất của dự án là `START_HERE.md`.
- Mã nguồn được tổ chức trong thư mục `src/` và kiểm thử trong `tests/`.
- CSDL hiện chưa được hiện thực; B-005 sẽ chốt đầy đủ schema. Các quyết định đã đóng băng yêu cầu tối thiểu `MetricSamples`, `ProcessedBatches` và cột `RowVersion` cho sự cố.

ĐOẠN NÀY CẦN NHÌN ĐIỀU GÌ?

- Cấu trúc thư mục gọn gàng, tách biệt giữa giao thức dùng chung và các ứng dụng thành phần.

```
LabGuard/
|-- START_HERE.md              # Huong dan nhap mon danh cho thanh vien
|-- FILE_MAP.md                # Tra cuu muc dich cua tung file trong repo
|-- database/
|   |-- schema.sql             # Vi tri script CSDL (placeholder)
|   |-- seed.sql               # Vi tri du lieu mau (placeholder)
|-- docs/
|   |-- pdf/                   # 5 cuon so tay huong dan PDF
|   `-- spec/                  # Dac ta giao thuc va quyet dinh kien truc
|-- src/
|   |-- LabGuard.Protocol/     # [DA HOAN THANH - 100%]
|   |-- LabGuard.Agent/        # [CHUA LAM / DU KIEN - Thanh vien A]
|   |-- LabGuard.Central/      # [CHUA LAM / DU KIEN - Thanh vien B]
|   |-- LabGuard.Admin/        # [CHUA LAM / DU KIEN - Thanh vien C]
|   `-- LabGuard.Simulator/    # [CHUA LAM / DU KIEN - Thanh vien D]
`-- tests/
    `-- LabGuard.Protocol.Tests/ # [DA HOAN THANH - Protocol tests PASS]
```

Listing 5: Cấu trúc thư mục thực tế của repository LabGuard

9.1 Các bảng/chức năng CSDL đã được chốt ở mức kiến trúc

- **MetricSamples:** Lưu mẫu CPU, RAM và `UptimeSeconds`; schema chi tiết do B-005 hiện thực.
- **ProcessedBatches:** Ghi khóa `(DeviceId, BootId, BatchId)` để nhận biết `METRIC_BATCH` gửi lại và chỉ `BATCH_ACK` sau khi commit thành công.
- **Incidents:** Dùng `RowVersion` do SQL Server tự sinh để phát hiện thao tác ghi đè giữa nhiều Admin.
- Các bảng nghiệp vụ còn lại (thiết bị, tài khoản, audit...) sẽ được chốt đầy đủ khi Thành viên B thực hiện B-005; tài liệu này không tự ấn định số bảng cuối cùng.

SAU PHẦN NÀY, BẠN CHỈ CẦN NHỚ:

- Khi bắt đầu, chỉ cần mở `START_HERE.md` là nắm được toàn bộ cách chạy dự án.

- Bạn B chịu trách nhiệm chạy script CSDL trong thư mục `database/`.

10. Phương pháp 5 câu hỏi đọc hiểu code C#

TÓM TẮT 30 GIÂY

- Dành cho sinh viên năm 2 chưa quen đọc mã nguồn dự án lớn.
- Trả lời 5 câu hỏi trước khi gõ phím để không bao giờ bị lạc lối trong code.

Khi bạn mở một tệp mã nguồn C# trong đồ án LabGuard, hãy áp dụng phương pháp 5 câu hỏi dưới đây:

1. **Lớp này sinh ra để làm gì?** Ví dụ: `FrameCodec` chỉ thêm/bớt 4 byte độ dài, không quan tâm nội dung bên trong là gì.
2. **Đoạn mã này chạy trên Luồng (Thread) nào?** Tác vụ mạng chạy trên ThreadPool ngầm; tương tác giao diện WinForms phải chạy trên UI Thread.
3. **Nếu mạng bị đứt tại dòng lệnh này thì sao?** Mã nguồn phải bắt ngoại lệ `SocketException` và kích hoạt cơ chế kết nối lại.
4. **Bộ nhớ RAM được giải phóng thế nào?** Luôn bọc `TcpClient`, `NetworkStream` trong khối lệnh `using`.
5. **Dữ liệu này đến từ đâu và đi về đâu?** Định vị vị trí của gói tin trong hành trình 7 bước giữa Agent, Central và Admin.

SAU PHẦN NÀY, BẠN CHỈ CẦN NHỚ:

- Luôn tự hỏi: ”Nếu dây mạng bị rút lúc này thì code của mình có bị văng lỗi không?”
- Luôn giải phóng tài nguyên mạng bằng từ khóa `using`.

11. Bốn thành viên phụ trách những phần việc gì?

TÓM TẮT 30 GIÂY

- Nhóm 4 người có ranh giới công việc cực kỳ mạch lạc.
- Mỗi thành viên có một cuốn cẩm nang riêng (Tập 2, 3, 4, 5) hướng dẫn từng bước.

- **Thành viên A (Protocol & Agent):** Xây dựng thư viện giao thức và lập trình Agent đo CPU/RAM, gửi nhịp tim và lưu bộ đệm khi rớt mạng.
- **Thành viên B (Central Server & CSDL):** Lập trình máy chủ TCP cổng 5000, kết nối SQL Server, phát hiện máy tắt (> 20s) và quá tải ($\geq 90\%$).
- **Thành viên C (Admin Dashboard):** Thiết kế giao diện WinForms, đồng bộ dữ liệu an toàn đa luồng (`BeginInvoke`) và xử lý khóa lạc quan OCC.
- **Thành viên D (Gateway & Collector):** Xây dựng tầng mở rộng quy mô (trạm trung chuyển Gateway và bộ thu gom Collector), giúp hệ thống phục vụ nhiều Agent và tự phục hồi khi mất

kết nối. (Ghi chú: Simulator và Wireshark là các công cụ hỗ trợ kiểm thử tải và phân tích gói tin).

SAU PHẦN NÀY, BẠN CHỈ CẦN NHỚ:

- Mỗi bạn tập trung vào cẩm nang riêng của mình và phối hợp qua cổng TCP 5000.

12. Hành trình phát triển theo 6 chặng công việc

TÓM TẮT 30 GIÂY

- Dự án được triển khai qua 6 chặng từ dễ đến khó.
- Chặng 1 đã xong 100%; nhóm đang ở Chặng 2 (kết nối đơn Agent ↔ Central).

Hình này chỉ cần nhìn theo các chặng từ trên xuống dưới: hoàn thành từng chặng một cách vững chắc trước khi chuyển sang chặng tiếp theo.

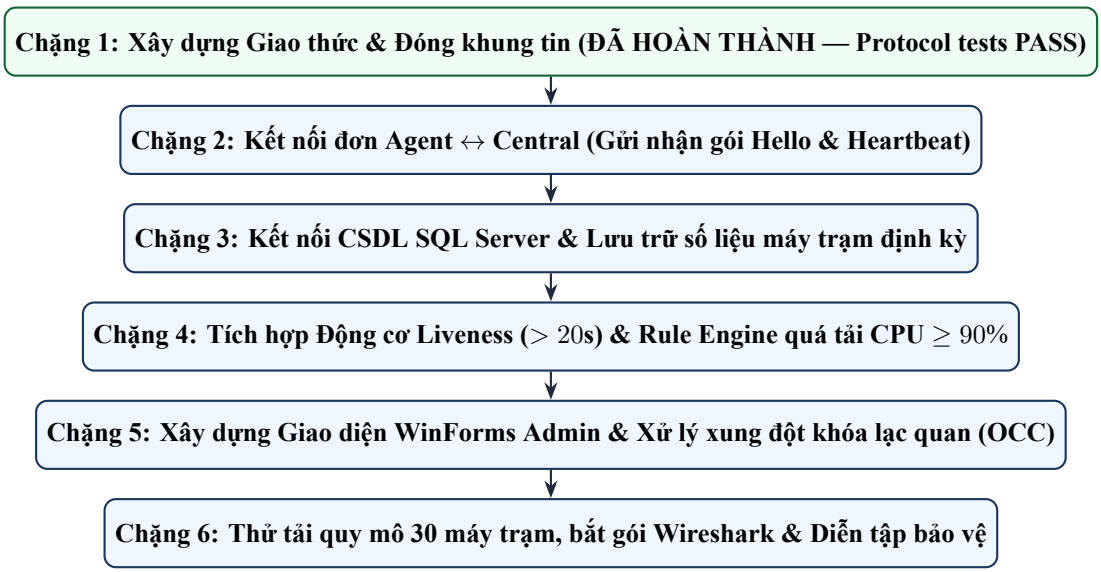


Figure 6: Lộ trình 6 chặng phát triển của đồ án LabGuard

Ý chính: Cách tiếp cận từng bước giúp cả nhóm luôn kiểm soát được tiến độ và không bị ngợp.

SAU PHẦN NÀY, BẠN CHỈ CẦN NHỚ:

- Nhiệm vụ trước mắt của nhóm là Chặng 2: cho Agent gửi được nhịp tim lên Central Server.

13. Quy trình làm việc nhóm an toàn với Git

TÓM TẮT 30 GIÂY

- Nhánh `main` luôn luôn biên dịch thành công.
- Mỗi người làm trên nhánh riêng, chạy `dotnet test` trước khi push code.

Để tránh xung đột mã nguồn khi 4 người cùng lập trình:

- Mỗi thành viên tạo một nhánh riêng: ví dụ `feature/agent-socket`, `feature/central-listener`.
- Trước khi đẩy code, bắt buộc chạy lệnh kiểm thử tự động `dotnet test`.
- Chỉ merge vào nhánh chính khi các bài kiểm thử đều đạt 100%.

SAU PHẦN NÀY, BẠN CHỈ CẦN NHỚ:

- Tuyệt đối không commit thẳng code lỗi lên nhánh `main`.

14. Cách biên dịch và kiểm thử dự án hiện tại

TÓM TẮT 30 GIÂY

- Biên dịch bằng lệnh `dotnet build LabGuard.sln`.
- Kiểm thử bằng lệnh `dotnet test tests/LabGuard.Protocol.Tests`.
- Thư viện Protocol đã có bộ kiểm thử tự động PASS hoàn toàn.

ĐOẠN NÀY CẦN NHÌN ĐIỀU GÌ?

- Lệnh build xác nhận mã nguồn không có lỗi cú pháp.

```
dotnet build LabGuard.sln
```

ĐOẠN NÀY CẦN NHÌN ĐIỀU GÌ?

- Lệnh chạy bộ kiểm thử tự động của thư viện Protocol.

```
dotnet test tests/LabGuard.Protocol.Tests/LabGuard.Protocol.Tests.csproj
```

Kết quả mong đợi:

```
Passed!  - Failed: 0, Passed: 107, Skipped: 0, Total: 107, Duration: 412 ms
```

SAU PHẦN NÀY, BẠN CHỈ CẦN NHỚ:

- Hãy tự mình gõ hai lệnh trên để chứng kiến toàn bộ bài test tự động đã PASS trên máy tính của bạn!

15. Luồng hoạt động khi hoàn thành toàn bộ đồ án (Dự kiến)

TÓM TẮT 30 GIÂY

- 4 biểu đồ tuần tự trực quan thể hiện toàn bộ kịch bản vận hành của đồ án.
- Luồng 1: Thu thập số liệu định kỳ; Luồng 2: Phát hiện mất kết nối sau 20s.
- Luồng 3: Động cơ quy tắc quá tải CPU; Luồng 4: Giải quyết xung đột khóa lạc quan OCC.

15.1 Thu thập số liệu định kỳ và cập nhật Dashboard

Agent trên máy tính sinh viên định kỳ gửi số liệu về Central. Central ghi vào SQL Server và đẩy thông tin mới nhất lên màn hình Admin Dashboard:

Hình này chỉ cần nhìn theo thứ tự số đánh dấu 1 đến 5: từ Agent gửi tin đến Central, Central lưu CSDL rồi báo sang Admin.

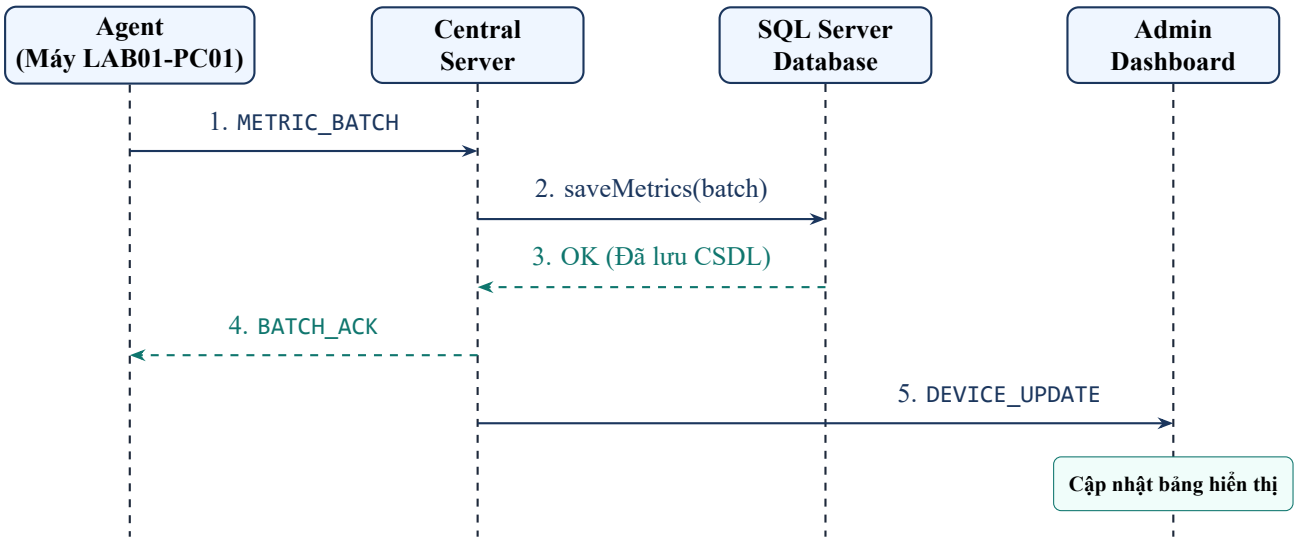


Figure 7: Biểu đồ tuần tự: Thu thập và cập nhật số liệu định kỳ

Ý chính: Số liệu đo đạc được lưu bền vững vào CSDL trước khi phản hồi xác nhận cho máy trạm.

15.2 Phát hiện mất kết nối mạng (Liveness Timeout)

Máy **`LAB01-PC02`** gửi gói `HEARTBEAT` định kỳ mỗi 5 giây. Nếu cáp mạng bị rút hoặc máy bị cúp điện, Central phát hiện máy im lặng quá 20 giây (> 20s) và kích hoạt sự cố:

Hình này chỉ cần nhìn vào ô sự cố màu đỏ ở giữa: sau hơn 20 giây không có nhịp tim, Central chuyển máy sang OFFLINE và báo cho Admin.

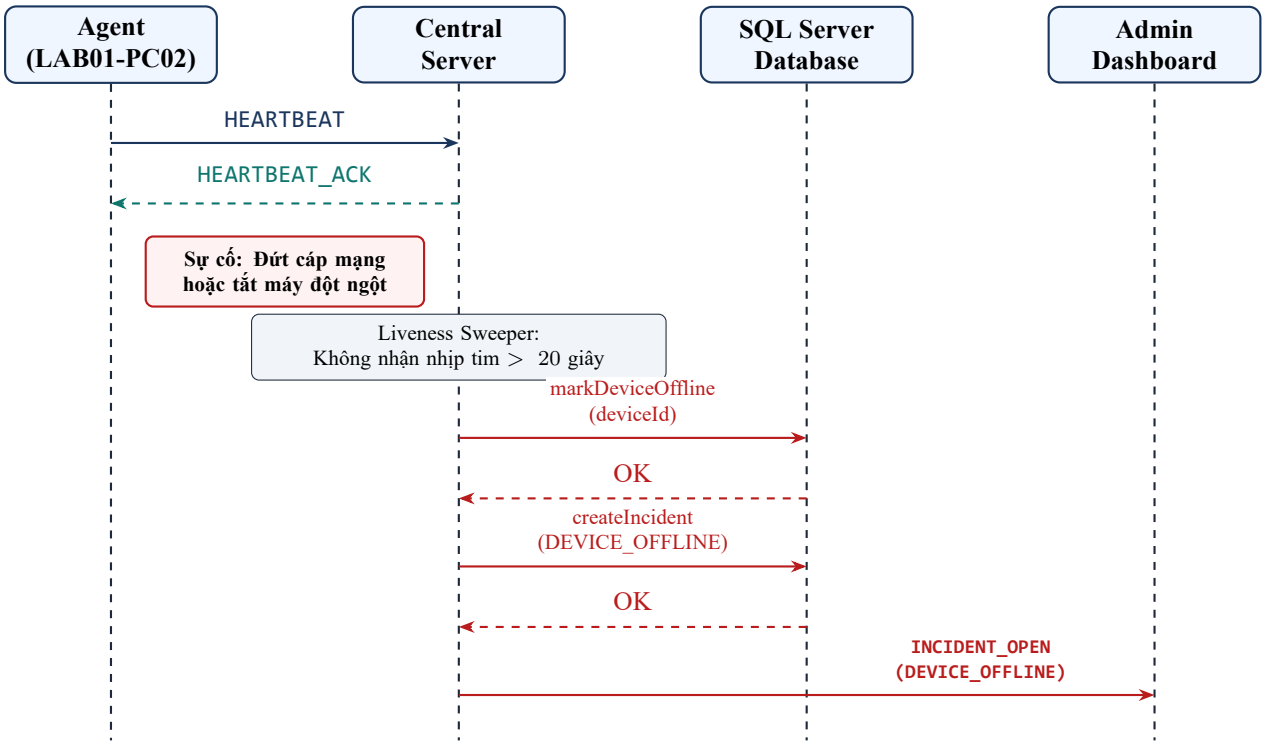


Figure 8: Biểu đồ tuần tự: Phát hiện mất kết nối mạng khi im lặng $>$ 20 giây

Ý chính: Ngưỡng 20 giây (gấp 4 lần chu kỳ 5s) giúp triệt tiêu hoàn toàn các cảnh báo giả do chập chờn mạng nhất thời.

15.3 Phát hiện quá tải CPU và tự phục hồi (RuleEngine)

Khi máy **`LAB01-PC02`** duy trì CPU $\geq$ 90% trong 3 mẫu đo liên tiếp, hệ thống kích hoạt sự cố `HIGH_CPU`. Khi CPU hạ xuống $\leq$ 75% trong 3 mẫu liên tiếp, Central Server tự động đóng sự cố và thông báo cho người quản trị.

15.4 Giải quyết xung đột khóa lạc quan (OCC Conflict Resolution)

Khi hai quản trị viên cùng thao tác trên một sự cố với cùng phiên bản ban đầu:

Hình này chỉ cần nhìn vào hai luồng thao tác: Admin 1 bấm trước thì thành công (200 OK), Admin 2 bấm sau thì bị từ chối (409 Conflict) để tránh ghi đè dữ liệu.

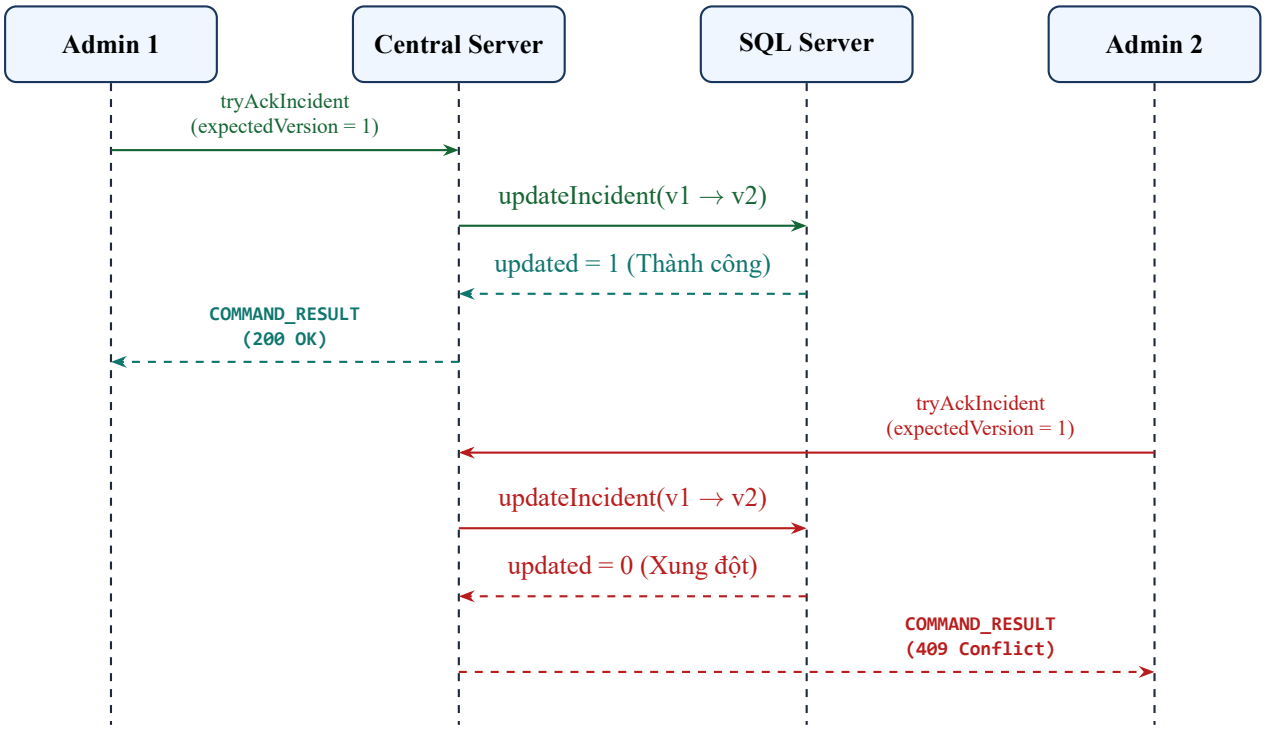


Figure 9: Biểu đồ tuần tự: Giải quyết xung đột ghi đồng thời bằng Khóa lạc quan (OCC)

Ý chính: Khóa lạc quan OCC bảo đảm dữ liệu sự cố luôn nhất quán mà không cần khóa cứng bảng CSDL.

SAU PHẦN NÀY, BẠN CHỈ CẦN NHỚ:

- 4 biểu đồ tuần tự là bản đồ chỉ đường cho toàn bộ giai đoạn hiện thực mã nguồn của nhóm.

16. Bảng tra cứu thuật ngữ mạng cốt lõi

TÓM TẮT 30 GIÂY

- Bảng tra cứu nhanh các thuật ngữ mạng cốt lõi xuất hiện trong đồ án.
- Dịch nghĩa tiếng Việt trực quan kèm giải thích ngắn gọn, dễ nhớ.

Table 4: Bảng thuật ngữ mạng máy tính áp dụng trong đồ án LabGuard

Thuật ngữ	Tên tiếng Anh	Ý nghĩa trong hệ thống LabGuard
TCP Stream	Byte Stream	Đặc tính truyền dữ liệu liên tục như dòng nước chảy của TCP, không tự có ranh giới gói.
Đóng khung	Packet Framing	Gắn 4 byte độ dài Big-Endian phía trước để bên nhận biết điểm kết thúc của gói tin.
Big-Endian	Network Byte Order	Thứ tự byte mạng chuẩn quốc tế, trong đó byte có trọng số lớn nhất được gửi trước.
Nhịp tim	Keepalive / Heartbeat	Gói tin định kỳ 5 giây giữa Agent và Central để chứng minh máy trạm còn sống.
Sống còn	Liveness Monitor	Bộ kiểm tra sống còn trên máy chủ, cảnh báo khi máy trạm im lặng quá 20 giây ($>$ 20s).
Khóa lạc quan	OCC	Kiểm soát tương tranh dựa trên cột `RowVersion`, báo lỗi 409 Conflict khi bị tranh chấp.
Lùi thời gian	Exponential Backoff	Tăng dần thời gian chờ khi thử kết nối lại (1s, 2s, 4s...) để tránh làm nghẽn máy chủ.
Phong bì mạng	WireEnvelope	Vỏ bọc dữ liệu 6 trường bắt buộc chuẩn hóa mọi trao đổi thông tin trong hệ thống.

SAU PHẦN NÀY, BẠN CHỈ CẦN NHỚ:

- Khi vấn đáp, hãy dùng thuật ngữ tiếng Việt trước, kèm theo tên tiếng Anh trong ngoặc đơn.

17. Bộ câu hỏi vấn đáp bảo vệ đồ án NT106

TÓM TẮT 30 GIÂY

- 30 câu hỏi thường gặp nhất khi bảo vệ đồ án môn Lập trình mạng.
- Đầy đủ câu trả lời chuẩn xác, súc tích, phục vụ tốt cho buổi vấn đáp bảo vệ đồ án.

Câu 1: Tại sao hệ thống sử dụng giao thức TCP thay vì UDP?
Trả lời: TCP cung cấp luồng truyền dữ liệu tin cậy (Reliable Byte Stream), không mất gói, đúng thứ tự và có cơ chế kiểm soát luồng. Dữ liệu giám sát phòng máy và các lệnh quản trị đòi hỏi độ chính xác tuyệt đối, do đó TCP là lựa chọn bắt buộc.

Câu 2: Hiện tượng dính gói và xé gói trên TCP là gì? Nhóm giải quyết bằng cách nào?
Trả lời: TCP là dòng byte liên tục, nhiều lệnh gửi có thể bị gộp vào một gói (dính gói) hoặc một bản tin bị chia cắt thành nhiều mẩu (xé gói). Nhóm giải quyết bằng kỹ thuật đóng khung nhị phân tiền tố độ dài (`FrameCodec`): gắn 4 byte Big-Endian chỉ độ dài payload phía trước mỗi nội dung JSON.

Câu 3: Tại sao lại chọn thứ tự byte Big-Endian cho 4 byte độ dài?
Trả lời: Big-Endian là chuẩn Network Byte Order theo quy chuẩn RFC 1700 của mạng Internet, bảo đảm mọi kiến trúc chip phần cứng khác nhau đều hiểu đúng cùng một giá trị độ dài.

Câu 4: Giới hạn kích thước khung tin của LabGuard là bao nhiêu? Tại sao?
Trả lời: Giới hạn là 262,144 byte (256 KiB), hoàn toàn dư dả cho các bản tin giám sát và

giúp ngăn chặn kẻ xấu gửi gói tin khổng lồ làm tràn bộ nhớ RAM máy chủ.

Câu 5: Cấu trúc WireEnvelope gồm có những trường nào?
Trả lời: Đúng 6 trường cố định: `version` (int = 1), `messageType` (string), `requestId` (GUID string), `deviceId` (string), `sentAtUtc` (ISO 8601 với offset UTC bằng 0: `Z` hoặc `+00:00`), và `payload` (generic T).

Câu 6: Chu kỳ gửi Heartbeat là bao lâu và thời gian xác định mất kết nối là bao nhiêu?
Trả lời: Chu kỳ gửi `HEARTBEAT` là 5 giây một lần. Thời gian xác định mất kết nối (Liveness Timeout) là khi máy trạm im lặng hơn 20 giây (> 20s), tương đương bỏ lỡ 4 chu kỳ nhịp tim liên tiếp.

Câu 7: Tại sao không dùng DateTime.UtcNow để tính thời gian mất kết nối mà nên dùng Stopwatch?
Trả lời: `DateTime.UtcNow` có thể bị nhảy khi máy chủ đồng bộ giờ NTP hoặc chỉnh giờ. `Stopwatch.GetTimestamp()` là đồng hồ đơn điệu (Monotonic Clock) luôn tăng đều, bảo đảm đo khoảng thời gian chính xác tuyệt đối.

Câu 8: Cơ chế Khóa lạc quan (OCC) hoạt động như thế nào trong LabGuard?
Trả lời: Bảng `Incidents` có cột `RowVersion`. Khi cập nhật trạng thái sự cố, câu lệnh SQL kiểm tra:

```
UPDATE Incidents SET WorkflowState = @NewState
OUTPUT inserted.RowVersion
WHERE IncidentId = @Id AND RowVersion = @ExpectedVer;
```

Nếu số dòng cập nhật được bằng 1 thì thành công (`200 OK`). Nếu bằng 0 nghĩa là đã có người sửa trước, server trả về mã lỗi `409 Conflict` để giao diện tải lại dữ liệu mới nhất.

Câu 9: Hiện tại mã nguồn của dự án đã làm được những gì?
Trả lời: Nhóm đã hoàn thành 100% phân hệ giao thức dùng chung `LabGuard.Protocol` (A-001 & A-002, 107/107 tests PASS) và tầng kết nối mạng của Agent (A-003: `AgentSocketClient.c` hàng đợi Single-Writer 256, 16/16 tests PASS). Các tác vụ tiếp theo của Agent (A-004+ như đo CPU/RAM, ReplayBuffer) đang trong kế hoạch thực hiện. Các phân hệ Central và Admin hiện là khung sườn tối thiểu để sinh viên B và C tự tay xây dựng.

Câu 10: Tại sao toàn bộ code mẫu viết sẵn trước đây lại bị dọn dẹp khỏi dự án?
Trả lời: Để bảo đảm tính liêm chính học thuật, giúp 4 bạn sinh viên tự tay lập trình từng dòng lệnh, hiểu sâu môn học và chịu trách nhiệm độc lập trên phân hệ của mình.

Câu 11: Máy chủ Central lắng nghe trên cổng nào?
Trả lời: Máy chủ Central mở cổng TCP 5000 duy nhất cho toàn bộ hệ thống. Máy chủ đọc bản tin đầu tiên sau khi kết nối (`HELLO_AGENT` hoặc `LOGIN`) để nhận diện và điều phối xử lý tương ứng.

Câu 12: Khi mất mạng, Agent lưu trữ dữ liệu như thế nào để không bị mất?
Trả lời: Theo thiết kế hệ thống (tác vụ A-006 dự kiến triển khai), Agent sẽ sử dụng bộ đệm ngoại tuyến (`ReplayBuffer`) lưu tối đa 50 mẻ số liệu (`METRIC_BATCH`, tương đương 250 giây dữ liệu) theo chính sách FIFO Drop-Oldest (loại bỏ mẻ cũ nhất khi đầy để bảo vệ bộ nhớ RAM). Khi mạng phục hồi, Agent tự động gửi bù các mẻ này kèm cờ `isReplay = true`. Lưu ý rằng bản tin nhịp tim (`HEARTBEAT`) và trạng thái dịch vụ (`SERVICE_STATUS`) không được lưu vào bộ đệm vì dữ liệu thời gian thực cũ không còn giá trị liveness.

Câu 13: Quy tắc cảnh báo quá tải CPU hoạt động ra sao?
Trả lời: Động cơ quy tắc yêu cầu mức CPU $\geq 90\%$ liên tục trong 3 mẫu đo (sau 3 chu kỳ

đo liên tiếp, mỗi chu kỳ 5 giây) để kích hoạt sự cố `HIGH_CPU`, và yêu cầu CPU $\leq 75\%$ liên tục trong 3 mẫu để tự động giải quyết sự cố.

Câu 14: Lỗi Cross-thread trên WinForms là gì? Khắc phục thế nào?
Trả lời: Là ngoại lệ xảy ra khi một luồng nền nhận tin TCP cố tình gán thuộc tính Control của giao diện. Khắc phục bằng cách kiểm tra `InvokeRequired` và gọi `BeginInvoke` để chuyển hành động về cho UI Thread thực hiện an toàn.

Câu 15: Bộ kiểm thử tự động của Protocol có bao nhiêu bài test?
Trả lời: Đúng 107 bài test tự động, kiểm thử bao phủ toàn bộ các trường hợp hợp lệ, trường hợp biên, đóng khung tin nhị phân và dữ liệu sai hỏng.

Câu 16: Tại sao chuỗi thời gian ISO 8601 bắt buộc phải có chỉ định UTC tường minh ('Z')?
Trả lời: Ký tự 'Z' đại diện cho giờ chuẩn quốc tế UTC, loại bỏ hoàn toàn nguy cơ sai lệch thời gian khi máy trạm và máy chủ đặt ở các múi giờ khác nhau.

Câu 17: Làm thế nào để sinh một RequestId chuẩn?
Trả lời: Sử dụng phương thức `Guid.NewGuid().ToString("D")`, sinh ra chuỗi 36 ký tự gồm chữ và số phân tách bằng 4 dấu gạch nối.

Câu 18: Tại sao không dùng Dictionary thông thường để lưu kết nối trên Central?
Trả lời: Vì `Dictionary` thông thường không an toàn đa luồng. Central phải dùng `ConcurrentDiction` để nhiều luồng truy cập cùng lúc không bị sập chương trình.

Câu 19: Bộ nhớ đệm FrameCodec có tối ưu được không?
Trả lời: Có thể sử dụng `ArrayPool<byte>.Shared` để mượn và trả mảng byte, giúp giảm tải công việc dọn rác (GC) của .NET khi lưu lượng gói tin lớn.

Câu 20: Thuật toán Exponential Backoff được cài đặt thế nào?
Trả lời: Khi mất kết nối, Agent chờ lần lượt 1s, 2s, 4s, 8s... tối đa 30 giây để tránh làm ngập máy chủ khi có sự cố diện rộng.

Câu 21: Cơ sở dữ liệu của LabGuard hiện tại đã hiện thực chưa và dự kiến gồm những gì?
Trả lời: CSDL chưa được hiện thực. Số bảng cuối cùng sẽ được chốt tại nhiệm vụ thiết kế schema của Bạn B. Kiến trúc hiện đã chốt các yêu cầu cốt lõi như `MetricSamples`, `ProcessedBatches` và `Incidents.RowVersion`; số bảng đầy đủ sẽ được chốt khi thực hiện B-005.

Câu 22: Làm thế nào kiểm tra xem một cổng mạng đã mở hay chưa bằng PowerShell?
Trả lời: Sử dụng lệnh `Test-NetConnection -ComputerName 127.0.0.1 -Port 5000`.

Câu 23: Tại sao Agent không kết nối trực tiếp vào CSDL SQL Server?
Trả lời: Để bảo mật thông tin xác thực, tránh làm lộ chuỗi kết nối và ngăn sinh viên can thiệp trực tiếp vào bảng dữ liệu điểm số, thông số phòng máy.

Câu 24: Phần mềm Wireshark giúp ích gì trong việc phát triển đồ án?
Trả lời: Cho phép quan sát thực tế quá trình bắt tay 3 bước TCP, cấu trúc 4 byte Big-Endian trên luồng mạng và giải mã nội dung JSON theo thời gian thực.

Câu 25: Khi sinh viên rút cáp mạng máy trạm, sự kiện gì diễn ra trên máy chủ?
Trả lời: Không có gói tin báo tắt nào gửi đến máy chủ. Máy chủ chỉ phát hiện được khi luồng quét `LivenessMonitor` nhận thấy máy tính đã im lặng hơn 20 giây (> 20s).

Câu 26: Tại sao phải dùng hàm ReadExactlyBytesAsync thay vì Stream.Read thông thường?
Trả lời: Vì TCP là dòng byte phân mảnh, một lệnh `Read` có thể chỉ nhận được 1–2 byte đầu tiên. Thuật toán `ReadExactlyBytesAsync` sử dụng vòng lặp gom tích lũy đủ số byte yêu cầu mới trả về, ngăn ngừa hoàn toàn lỗi đọc thiếu dữ liệu.

Câu 27: Sự khác biệt giữa Invoke và BeginInvoke trong WinForms là gì?
Trả lời: `Invoke` là gọi đồng bộ, luồng mạng bị chặn đứng chờ UI vẽ xong. `BeginInvoke` là gọi bất đồng bộ, đẩy tác vụ vào hàng đợi của UI rồi luồng mạng tiếp tục đọc gói tin ngay lập tức, không gây giật lag.

Câu 28: Làm thế nào để chống tấn công SQL Injection trong Central Server?
Trả lời: Luôn sử dụng truy vấn tham số hóa (Parameterized Queries) thông qua `SqlCommand.Paramete` tuyệt đối không bao giờ cộng chuỗi lệnh SQL.

Câu 29: Nếu máy chủ bị khởi động lại, các kết nối TCP hiện tại sẽ ra sao?
Trả lời: Tất cả kết nối TCP cũ sẽ bị đứt. Khi máy chủ bật lại, các Agent tự động kết nối lại và gửi bản tin `HELLO_AGENT` để tái thiết lập phiên làm việc.

Câu 30: Thành công lớn nhất mà sinh viên học được qua đồ án này là gì?
Trả lời: Hiểu sâu sắc bản chất tầng Transport và Application trong mô hình OSI: cách xử lý luồng byte TCP, đóng khung tin nhị phân, thiết kế giao thức mạng độc lập và xử lý bất đồng bộ đa luồng trong một hệ thống phân tán thực tế.

— HẾT SỔ TAY TỔNG QUAN A–Z —